SNOWFLAKE CERTIFICATION STUDY SERIES

MODULE 2: ACCOUNT MANAGEMENT & GOVERNANCE

Domain 2.0 Deep-Dive Guide, Technical Specs & Lab Worksheets

SnowPro Advanced: Administrator (ADA-C02)

Specification Parameter	Detail / Blueprint Alignment
Domain Coverage	Domain 2.0: Account Management & Data Governance
Exam Weighting	18% of Total Scaled Score
Target Readiness	Production Governance, Multi-Account & Column/Row Security
Key Features	Organizations, Tri-Secret Secure, Dynamic Masking, Row Access Policies, Access History

Guide Blueprint: This manual synthesizes official Snowflake Data Governance and Account Administration architecture into actionable study notes, parameter boundaries, and hands-on lab worksheets. Execute all SQL labs in your trial worksheet to build required scenario muscle memory.

1. Organizations & Multi-Account Administration

Snowflake Organizations (`ORGANIZATION`) allow enterprises to manage multiple accounts across cloud providers (AWS, Azure, GCP) and regions under a single unified billing and governance boundary.

Key Administrative Capabilities:

Feature / Command	Administrative Description	Exam Boundary
Feature / Command	Administrative Description	Exam Boundary
ORGADMIN Role	System-defined role managing organization accounts.	Must be enabled explicitly by Snowflake Support or ACCOUNTADMIN.
CREATE ACCOUNT	Programmatically provisions new accounts.	Requires specify NAME, ADMIN_NAME, ADMIN_PASSWORD, EDITION, REGION.
SHOW ACCOUNTS	Lists all active/dropped accounts in org.	Displays account locator, cloud, region, and edition details.
Global URL Routing	orgname-accountname.snowflakecomputing.com	Allows seamless access regardless of underlying cloud region.

2. Data Encryption & Tri-Secret Secure (TSS)

Snowflake employs hierarchical end-to-end encryption. All customer data stored in Snowflake is automatically encrypted at rest using strong AES-256 keys managed in a key hierarchy.

Key Hierarchy Breakdown:

Level	Key Type	Description & Management
Level	Key Type	Description & Management
Root Key	HSM Master Key	Stored in Hardware Security Modules (HSMs) in Cloud.
Account Key	Account-Level Key	Encrypts database keys; rotated annually by Snowflake.
Database Key	Container Key	Encrypts table-level keys for all child tables.
Table Key	Object Key	Encrypts individual file micro-partitions at rest.

> **Tri-Secret Secure (TSS) Concept:** TSS combines a customer-managed key (in AWS KMS, Azure Key Vault, or GCP KMS) with a Snowflake-managed key to create a composite Master Key. If the customer revokes access to their KMS key, data access across the entire Snowflake account is immediately halted (Data Lockout).

3. Dynamic Data Masking & Column-Level Security

Dynamic Data Masking allows administrators to mask sensitive column data (PII, PHI, Financials) at query runtime based on the user's active session role without altering physical storage.

```
--- Create a reusable masking policy for SSN columns
CREATE OR REPLACE MASKING POLICY hr_db.policies.ssn_mask AS (val string) RETURNS string ->
  CASE
    WHEN CURRENT_ROLE() IN ('HR_ADMIN', 'ACCOUNTADMIN') THEN val
    ELSE 'XXX-XX-' || RIGHT(val, 4)
  END;

--- Apply policy to a sensitive column
ALTER TABLE hr_db.payroll.employees ALTER COLUMN ssn SET MASKING POLICY hr_db.policies.ssn_mask;
```

Masking Policy Evaluation Rules:

Rule / Clause	Behavioral Mechanic
Rule / Clause	Behavioral Mechanic
CURRENT_ROLE()	Evaluates the user's active primary role in the current session.
INVOKER_ROLE()	Evaluates the role executing a view or stored procedure calling masked data.
USING Clause	Enables Conditional Masking based on values in OTHER columns in the same row.
Unset Policy	ALTER TABLE tbl ALTER COLUMN col UNSET MASKING POLICY; (Requires APPLY MASKING POLICY privilege).

4. Row Access Policies (RAP) & Row-Level Security

Row Access Policies filter returned rows at query time based on user session context, role, or mapping table lookups. RAPs act as a transparent SQL predicate applied before query compilation.

```
--- Create Row Access Policy using a mapping table
CREATE OR REPLACE ROW ACCESS POLICY sales_db.policies.region_filter
AS (sales_region string) RETURNS BOOLEAN ->
  EXISTS (
    SELECT 1 FROM sales_db.policies.sales_role_mapping
    WHERE role_name = CURRENT_ROLE() AND region = sales_region
  );

--- Attach RAP to a Fact Table
ALTER TABLE sales_db.sales.orders ADD ROW ACCESS POLICY sales_db.policies.region_filter ON (order_region);
```

Combining RAPs & Dynamic Masking:

When both a Row Access Policy and a Dynamic Masking Policy are active on the same table, Snowflake evaluates the **Row Access Policy FIRST** to determine which rows the user is permitted to see, and then evaluates the **Masking Policy SECOND** to mask individual column values in those permitted rows.

5. Object Tagging, Sensitive Data Classification & Audit

Governance in Snowflake relies on metadata tagging, automated classification, and tracking via `ACCESS_HISTORY`.

Tool / Feature	Command / View	Key Administrator Function
Tool / Feature	Command / View	Key Administrator Function
Object Tagging	CREATE TAG cost_center;	Attaches key-value metadata to databases, warehouses, or tables for tracking.
Tag-Based Masking	APPLY MASKING POLICY ON TAG	Automatically applies a masking policy to any column assigned a specific tag.
Data Classification	SYSTEM$CLASSIFY()	Scans tables to automatically detect and tag PII/sensitive data.
Access History Audit	ACCOUNT_USAGE.ACCESS_HISTORY	Logs direct and base objects/columns accessed by user queries for compliance.

6. Hands-On Lab Worksheets (Module 2 Practice)

Lab 1: Implementing Dynamic Data Masking & Role Verification

```
USE ROLE ACCOUNTADMIN;
CREATE DATABASE IF NOT EXISTS GOV_DEMO_DB;
CREATE SCHEMA IF NOT EXISTS GOV_DEMO_DB.POLICIES;
CREATE SCHEMA IF NOT EXISTS GOV_DEMO_DB.DATA;

--- Create Test Table & Data
CREATE OR REPLACE TABLE GOV_DEMO_DB.DATA.CUSTOMERS (
  CUST_ID INT, NAME STRING, EMAIL STRING, CREDIT_CARD STRING
);
INSERT INTO GOV_DEMO_DB.DATA.CUSTOMERS VALUES
  (101, 'Alice Smith', 'alice@example.com', '4532-1122-3344-5566'),
  (102, 'Bob Jones', 'bob@example.com', '5412-8899-7766-1122');

--- Create Masking Policy
CREATE OR REPLACE MASKING POLICY GOV_DEMO_DB.POLICIES.CC_MASK AS (val string) RETURNS string ->
  CASE WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN', 'FINANCE_ADMIN') THEN val
       ELSE 'XXXX-XXXX-XXXX-' || RIGHT(val, 4) END;

--- Apply Policy
ALTER TABLE GOV_DEMO_DB.DATA.CUSTOMERS ALTER COLUMN CREDIT_CARD SET MASKING POLICY GOV_DEMO_DB.POLICIES.CC_

--- Verify as ACCOUNTADMIN
SELECT * FROM GOV_DEMO_DB.DATA.CUSTOMERS;
```

Lab 2: Building a Mapping-Table Row Access Policy (RAP)

```
--- Create Mapping Table
CREATE OR REPLACE TABLE GOV_DEMO_DB.POLICIES.ROLE_REGION_MAP (ROLE_NAME STRING, REGION STRING);
INSERT INTO GOV_DEMO_DB.POLICIES.ROLE_REGION_MAP VALUES ('SALES_US', 'US'), ('SALES_EU', 'EU');

--- Create Table
CREATE OR REPLACE TABLE GOV_DEMO_DB.DATA.REGIONAL_SALES (ORDER_ID INT, REGION STRING, AMOUNT NUMBER);
INSERT INTO GOV_DEMO_DB.DATA.REGIONAL_SALES VALUES (1, 'US', 5000), (2, 'EU', 3000), (3, 'ASIA', 4000);

--- Create Row Access Policy
CREATE OR REPLACE ROW ACCESS POLICY GOV_DEMO_DB.POLICIES.REGIONAL_RAP AS (sales_region string) RETURNS BOOL
  EXISTS (SELECT 1 FROM GOV_DEMO_DB.POLICIES.ROLE_REGION_MAP WHERE ROLE_NAME = CURRENT_ROLE() AND REGION =
  OR CURRENT_ROLE() = 'ACCOUNTADMIN';

--- Apply RAP to Table
ALTER TABLE GOV_DEMO_DB.DATA.REGIONAL_SALES ADD ROW ACCESS POLICY GOV_DEMO_DB.POLICIES.REGIONAL_RAP ON (REG
```

Lab 3: Tag-Based Masking Policy Automation

```
--- Create Tag
CREATE OR REPLACE TAG GOV_DEMO_DB.POLICIES.CONFIDENTIALITY_TAG ALLOWED_VALUES 'HIGH', 'LOW';

--- Assign Masking Policy to Tag
APPLY MASKING POLICY GOV_DEMO_DB.POLICIES.CC_MASK ON TAG GOV_DEMO_DB.POLICIES.CONFIDENTIALITY_TAG;

--- Apply Tag to Column
ALTER TABLE GOV_DEMO_DB.DATA.CUSTOMERS ALTER COLUMN EMAIL SET TAG GOV_DEMO_DB.POLICIES.CONFIDENTIALITY_TAG

--- Verify Tag References
SELECT * FROM TABLE(GOV_DEMO_DB.INFORMATION_SCHEMA.TAG_REFERENCES('GOV_DEMO_DB.DATA.CUSTOMERS.EMAIL', 'COLU
```